

LECTURE 7

CEN 348-Artificial Intelligence Systems

4.7 Evaluation: Precision, Recall, F-measure

To introduce the methods for evaluating text classification, let's first consider some simple binary *detection* tasks. For example, in spam detection, our goal is to label every text as being in the spam category (“positive”) or not in the spam category (“negative”). For each item (email document) we therefore need to know whether our system called it spam or not. We also need to know whether the email is actually spam or not, i.e. the human-defined labels for each document that we are trying to match. We will refer to these human labels as the **gold labels**.

Or imagine you're the CEO of the *Delicious Pie Company* and you need to know what people are saying about your pies on social media, so you build a system that detects tweets concerning Delicious Pie. Here the positive class is tweets about Delicious Pie and the negative class is all other tweets.

In both cases, we need a metric for knowing how well our spam detector (or pie-tweet-detector) is doing. To evaluate any system for detecting things, we start by building a **confusion matrix** like the one shown in Fig. 4.4. A confusion matrix is a table for visualizing how an algorithm performs with respect to the human gold labels, using two dimensions (system output and gold labels), and each cell labeling a set of possible outcomes. In the spam detection case, for example, true positives are documents that are indeed spam (indicated by human-created gold labels) that our system correctly said were spam. False negatives are documents that are indeed spam but our system incorrectly labeled as non-spam.

To the bottom right of the table is the equation for *accuracy*, which asks what percentage of all the observations (for the spam or pie examples that means all emails or tweets) our system labeled correctly. Although accuracy might seem a natural metric, we generally don't use it for text classification tasks. That's because accuracy doesn't work well when the classes are unbalanced (as indeed they are with spam, which is a large majority of email, or with tweets, which are mainly not about pie).

		<i>gold standard labels</i>		
		gold positive	gold negative	
<i>system output labels</i>	system positive	true positive	false positive	precision = $\frac{tp}{tp+fp}$
	system negative	false negative	true negative	
		recall = $\frac{tp}{tp+fn}$		accuracy = $\frac{tp+tn}{tp+fp+tn+fn}$

Figure 4.4 A confusion matrix for visualizing how well a binary classification system performs against gold standard labels.

- To make this more explicit, imagine that we looked at a million tweets, and let's say that only 100 of them are discussing their love (or hatred) for our pie, while the other 999,900 are tweets about something completely unrelated.
- Imagine a simple classifier that stupidly classified every tweet as “not about pie”. This classifier would have 999,900 true negatives and only 100 false negatives for an accuracy of $999,900/1,000,000$ or 99.99%!
- What an amazing accuracy level! Surely we should be happy with this classifier? But of course this fabulous ‘no pie’ classifier would be completely useless, since it wouldn't find a single one of the customer comments we are looking for.
- In other words, accuracy is not a good metric when the goal is to discover something that is rare, or at least not completely balanced in frequency, which is a very common situation in the world.

precision

That's why instead of accuracy we generally turn to two other metrics shown in Fig. 4.4: **precision** and **recall**. **Precision** measures the percentage of the items that the system detected (i.e., the system labeled as positive) that are in fact positive (i.e., are positive according to the human gold labels). Precision is defined as

$$\text{Precision} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

recall

Recall measures the percentage of items actually present in the input that were correctly identified by the system. Recall is defined as

$$\text{Recall} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

Precision and recall will help solve the problem with the useless “nothing is pie” classifier. This classifier, despite having a fabulous accuracy of 99.99%, has a terrible recall of 0 (since there are no true positives, and 100 false negatives, the recall is 0/100). You should convince yourself that the precision at finding relevant tweets is equally problematic. Thus precision and recall, unlike accuracy, emphasize true positives: finding the things that we are supposed to be looking for.

F-measure

There are many ways to define a single metric that incorporates aspects of both precision and recall. The simplest of these combinations is the **F-measure** (van Rijsbergen, 1975) , defined as:

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}$$

The β parameter differentially weights the importance of recall and precision, based perhaps on the needs of an application. Values of $\beta > 1$ favor recall, while values of $\beta < 1$ favor precision. When $\beta = 1$, precision and recall are equally balanced; this is the most frequently used metric, and is called $F_{\beta=1}$ or just F_1 :

F1

$$F_1 = \frac{2PR}{P + R} \quad (4.16)$$

4.7.1 Evaluating with more than two classes

Up to now we have been describing text classification tasks with only two classes. But lots of classification tasks in language processing have more than two classes. For sentiment analysis we generally have 3 classes (positive, negative, neutral) and even more classes are common for tasks like part-of-speech tagging, word sense disambiguation, semantic role labeling, emotion detection, and so on. Luckily the naive Bayes algorithm is already a multi-class classification algorithm.

		<i>gold labels</i>			
		urgent	normal	spam	
<i>system output</i>	urgent	8	10	1	$\text{precision}_u = \frac{8}{8+10+1}$
	normal	5	60	50	$\text{precision}_n = \frac{60}{5+60+50}$
	spam	3	30	200	$\text{precision}_s = \frac{200}{3+30+200}$
		$\text{recall}_u = \frac{8}{8+5+3}$	$\text{recall}_n = \frac{60}{10+60+30}$	$\text{recall}_s = \frac{200}{1+50+200}$	

Figure 4.5 Confusion matrix for a three-class categorization task, showing for each pair of classes (c_1, c_2) , how many documents from c_1 were (in)correctly assigned to c_2

macroaveraging
microaveraging

But we'll need to slightly modify our definitions of precision and recall. Consider the sample confusion matrix for a hypothetical 3-way *one-of* email categorization decision (urgent, normal, spam) shown in Fig. 4.5. The matrix shows, for example, that the system mistakenly labeled one spam document as urgent, and we have shown how to compute a distinct precision and recall value for each class. In order to derive a single metric that tells us how well the system is doing, we can combine these values in two ways. In **macroaveraging**, we compute the performance for each class, and then average over classes. In **microaveraging**, we collect the decisions for all classes into a single confusion matrix, and then compute precision and recall from that table. Fig. 4.6 shows the confusion matrix for each class separately, and shows the computation of microaveraged and macroaveraged precision.

As the figure shows, a microaverage is dominated by the more frequent class (in this case spam), since the counts are pooled. The macroaverage better reflects the statistics of the smaller classes, and so is more appropriate when performance on all the classes is equally important.

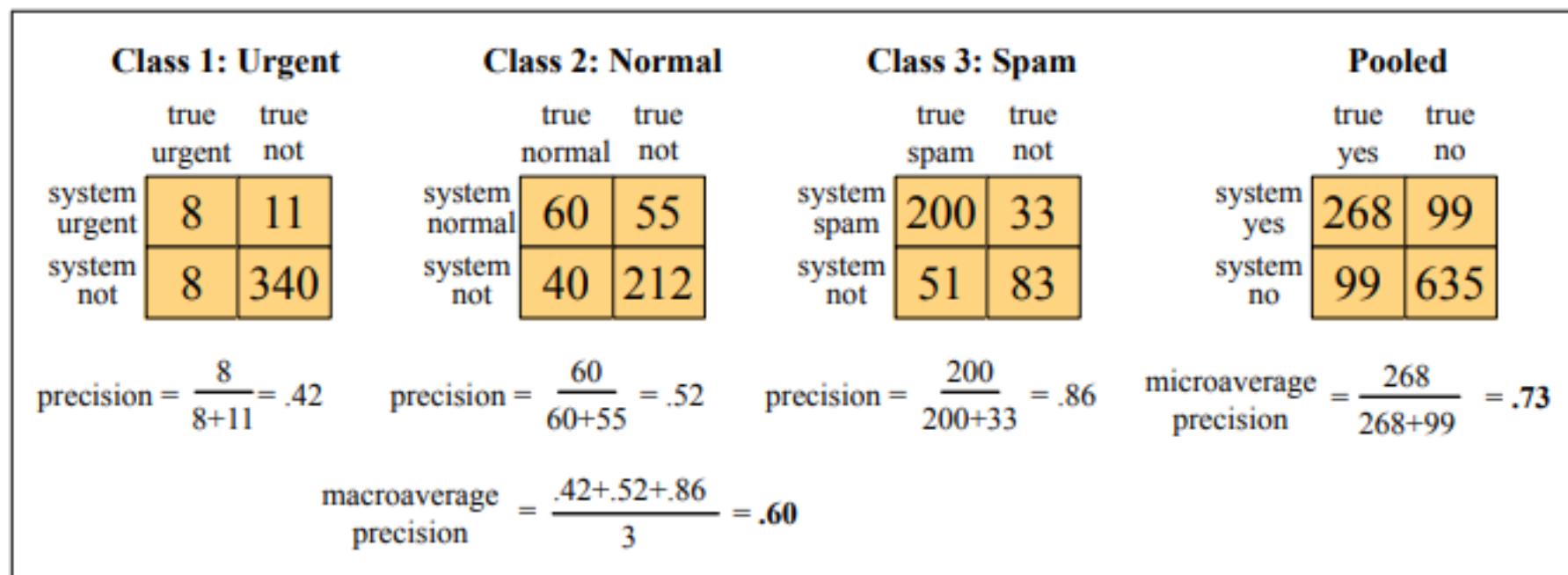


Figure 4.6 Separate confusion matrices for the 3 classes from the previous figure, showing the pooled confusion matrix and the microaveraged and macroaveraged precision.

Test sets and Cross-validation

- we use the training set to train the model, then use the development test set (also called a devset).
- While the use of a devset avoids overfitting the test set, having a fixed training set, devset, and test set creates another problem: in order to save lots of data for training, the test set (or devset) might not be large enough to be representative.
- Wouldn't it be better if we could somehow use all our data for training and still use all our data for test?

Test sets and Cross-validation

- We can do this by cross-validation: we randomly choose a training and test set division of our data, train our classifier, and then compute the error rate on the test set.
- Then we repeat with a different randomly selected training set and test set.
- We do this sampling process 10 times and average these 10 runs to get an average error rate. This is called 10-fold cross-validation.

Neural Networks

- Neural networks are a fundamental computational tool.
- They are called neural because their origins lie in the McCulloch-Pitts neuron (McCulloch and Pitts, 1943), a simplified model of the human neuron as a kind of computing element that could be described in terms of propositional logic.
- A modern neural network is a network of small computing units, each of which takes a vector of input values and produces a single output value.
- We introduce the neural net applied to classification.

Neural Networks

- The architecture we introduce is called a feedforward network because the computation proceeds iteratively from one layer of units to the next.
- The use of modern neural nets is often called deep learning, because modern networks are often deep (have many layers).
- Neural networks share much of the same mathematics as logistic regression.
- But neural networks are a more powerful classifier than logistic regression, and indeed a minimal neural network (technically one with a single 'hidden layer') can be shown to learn any function.

Units

- The building block of a neural network is a single computational unit.
- A unit takes a set of real valued numbers as input, performs some computation on them, and produces an output.
- At its heart, a neural unit is taking a weighted sum of its inputs, with one additional term in the sum called a **bias** term.
- Given a set of inputs $x_1 \dots x_n$, a unit has a set of corresponding weights $w_1 \dots w_n$ and a bias b , so the weighted sum z can be represented as:

$$z = b + \sum_i w_i x_i \quad (7.1)$$

Units

- Often it's more convenient to express this weighted sum using vector notation; recall from linear algebra that a vector is, at heart, just a list or array of numbers.
- Thus we'll talk about z in terms of a weight vector w , a scalar bias b , and an input vector x , and we'll replace the sum with the convenient dot product:
- As defined in Eq. 7.2, z is just a real valued number.
- Finally, instead of using z , a linear function of x , as the output, neural units apply a non-linear function f to z .

$$z = w \cdot x + b \quad (7.2)$$

Units

- We will refer to the output of this function as the activation value for the unit, a .
- Since we are just modeling a single unit, the activation for the node is in fact the final output of the network, which we'll generally call y . So the value y is defined as:

$$y = a = f(z)$$

We'll discuss three popular non-linear functions $f()$ below (the sigmoid, the tanh, and the rectified linear ReLU)

Units

$$y = \sigma(z) = \frac{1}{1 + e^{-z}} \quad (7.3)$$

The sigmoid (shown in Fig. 7.1) has a number of advantages; it maps the output into the range $[0, 1]$, which is useful in squashing outliers toward 0 or 1. And it's differentiable, which as we saw in Section 5.8 will be handy for learning.

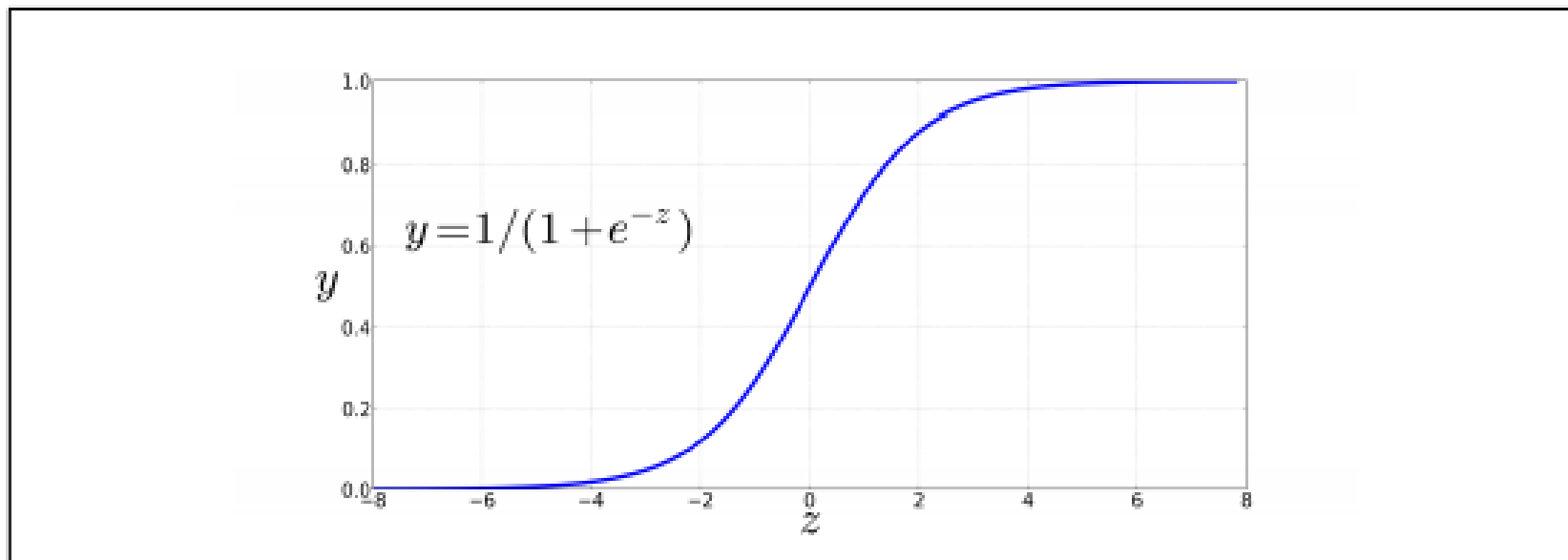


Figure 7.1 The sigmoid function takes a real value and maps it to the range $[0, 1]$. It is nearly linear around 0 but outlier values get squashed toward 0 or 1.

Units

Substituting Eq. 7.2 into Eq. 7.3 gives us the output of a neural unit:

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + \exp(-(w \cdot x + b))} \quad (7.4)$$

Fig. 7.2 shows a final schematic of a basic neural unit. In this example the unit takes 3 input values x_1, x_2 , and x_3 , and computes a weighted sum, multiplying each value by a weight (w_1, w_2 , and w_3 , respectively), adds them to a bias term b , and then passes the resulting sum through a sigmoid function to result in a number between 0 and 1.

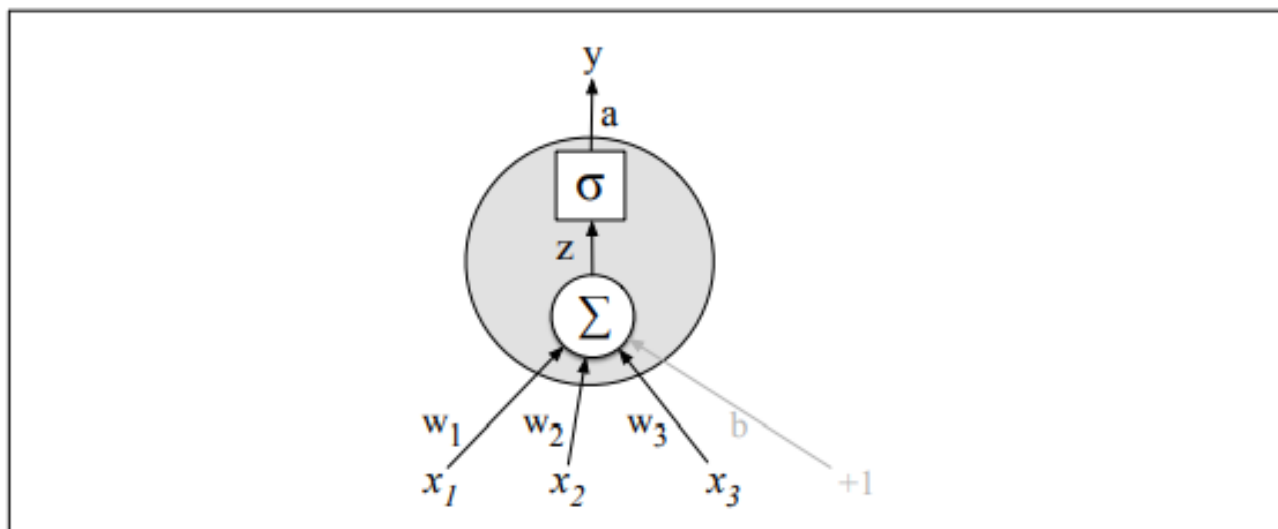


Figure 7.2 A neural unit, taking 3 inputs x_1, x_2 , and x_3 (and a bias b that we represent as a weight for an input clamped at $+1$) and producing an output y . We include some convenient intermediate variables: the output of the summation, z , and the output of the sigmoid, a . In this case the output of the unit y is the same as a , but in deeper networks we'll reserve y to mean the final output of the entire network, leaving a as the activation of an individual node.

Units

Let's walk through an example just to get an intuition. Let's suppose we have a unit with the following weight vector and bias:

$$w = [0.2, 0.3, 0.9]$$

$$b = 0.5$$

What would this unit do with the following input vector:

$$x = [0.5, 0.6, 0.1]$$

The resulting output y would be:

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} = \frac{1}{1 + e^{-(.5 \cdot .2 + .6 \cdot .3 + .1 \cdot .9 + .5)}} = \frac{1}{1 + e^{-0.87}} = .70$$

Units

tanh

In practice, the sigmoid is not commonly used as an activation function. A function that is very similar but almost always better is the **tanh** function shown in Fig. 7.3a; tanh is a variant of the sigmoid that ranges from -1 to +1:

$$y = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (7.5)$$

ReLU

The simplest activation function, and perhaps the most commonly used, is the rectified linear unit, also called the **ReLU**, shown in Fig. 7.3b. It's just the same as x when x is positive, and 0 otherwise:

$$y = \max(x, 0) \quad (7.6)$$

Units

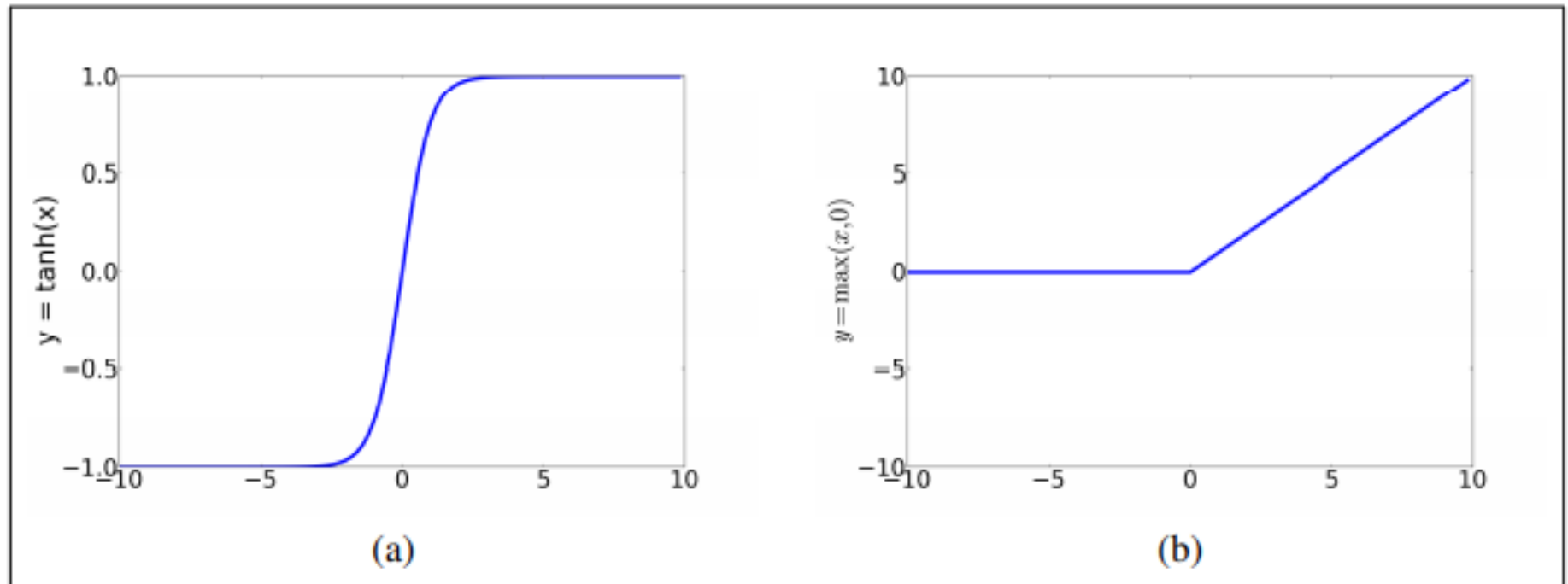


Figure 7.3 The tanh and ReLU activation functions.

The XOR problem

Early in the history of neural networks it was realized that the power of neural networks, as with the real neurons that inspired them, comes from combining these units into larger networks.

One of the most clever demonstrations of the need for multi-layer networks was the proof by [Minsky and Papert \(1969\)](#) that a single neural unit cannot compute some very simple functions of its input. Consider the task of computing elementary logical functions of two inputs, like AND, OR, and XOR. As a reminder, here are the truth tables for those functions:

AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

The XOR problem

perceptron

This example was first shown for the **perceptron**, which is a very simple neural unit that has a binary output and does **not** have a non-linear activation function. The output y of a perceptron is 0 or 1, and is computed as follows (using the same weight w , input x , and bias b as in Eq. 7.2):

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases} \quad (7.7)$$

It's very easy to build a perceptron that can compute the logical AND and OR functions of its binary inputs; Fig. 7.4 shows the necessary weights.

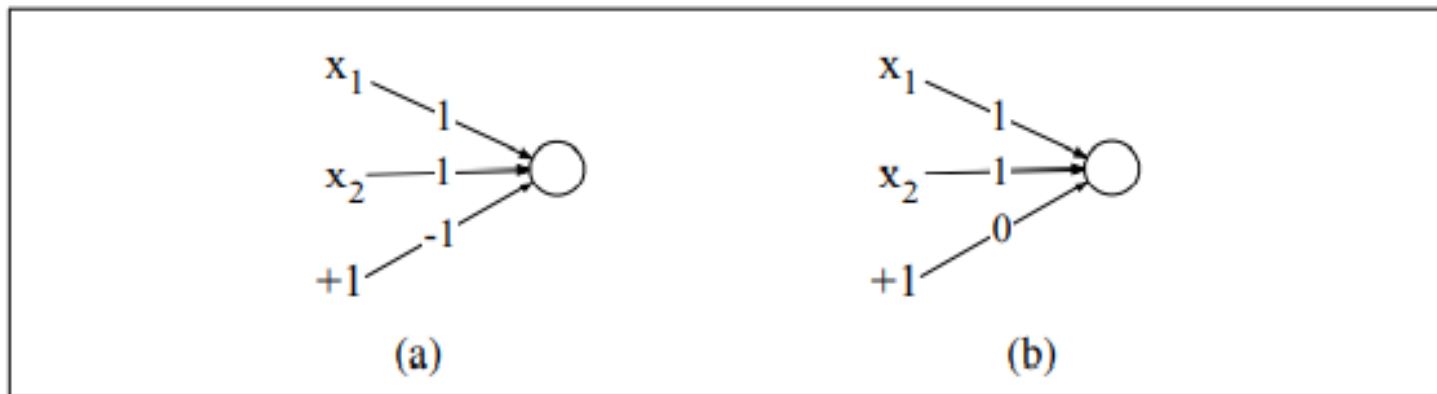


Figure 7.4 The weights w and bias b for perceptrons for computing logical functions. The inputs are shown as x_1 and x_2 and the bias as a special node with value $+1$ which is multiplied with the bias weight b . (a) logical AND, showing weights $w_1 = 1$ and $w_2 = 1$ and bias weight $b = -1$. (b) logical OR, showing weights $w_1 = 1$ and $w_2 = 1$ and bias weight $b = 0$. These weights/biases are just one from an infinite number of possible sets of weights and biases that would implement the functions.

The XOR problem

It turns out, however, that it's not possible to build a perceptron to compute logical XOR! (It's worth spending a moment to give it a try!)

The intuition behind this important result relies on understanding that a perceptron is a linear classifier. For a two-dimensional input x_1 and x_2 , the perception equation, $w_1x_1 + w_2x_2 + b = 0$ is the equation of a line. (We can see this by putting it in the standard linear format: $x_2 = (-w_1/w_2)x_1 + (-b/w_2)$.) This line acts as a **decision boundary** in two-dimensional space in which the output 0 is assigned to all inputs lying on one side of the line, and the output 1 to all input points lying on the other side of the line. If we had more than 2 inputs, the decision boundary becomes a hyperplane instead of a line, but the idea is the same, separating the space into two categories.

decision
boundary

The XOR problem

Fig. 7.5 shows the possible logical inputs (00, 01, 10, and 11) and the line drawn by one possible set of parameters for an AND and an OR classifier. Notice that there is simply no way to draw a line that separates the positive cases of XOR (01 and 10) from the negative cases (00 and 11). We say that XOR is not a **linearly separable** function. Of course we could draw a boundary with a curve, or some other function, but not a single line.

linearly
separable

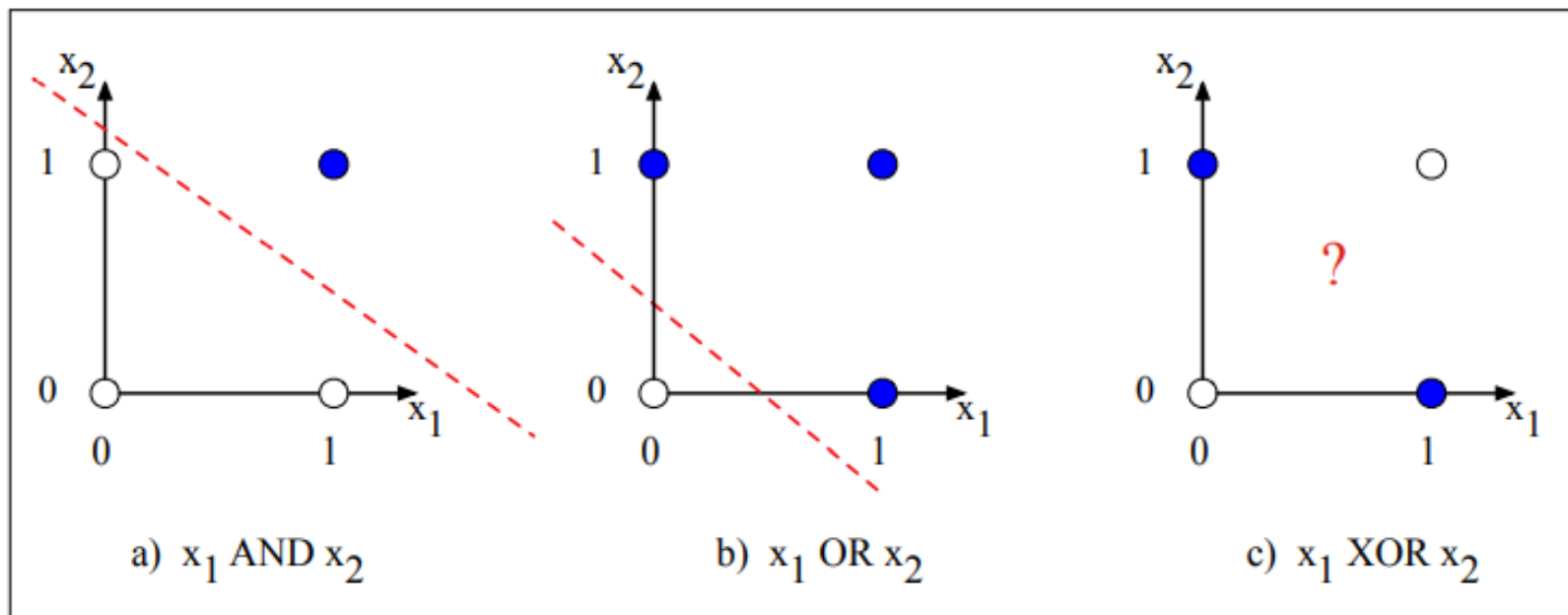


Figure 7.5 The functions AND, OR, and XOR, represented with input x_1 on the x-axis and input x_2 on the y axis. Filled circles represent perceptron outputs of 1, and white circles perceptron outputs of 0. There is no way to draw a line that correctly separates the two categories for XOR. Figure styled after [Russell and Norvig \(2002\)](#).

The solution: neural networks

While the XOR function cannot be calculated by a single perceptron, it can be calculated by a layered network of units. Let's see an example of how to do this from [Goodfellow et al. \(2016\)](#) that computes XOR using two layers of ReLU-based units. Fig. 7.6 shows a figure with the input being processed by two layers of neural units. The middle layer (called h) has two units, and the output layer (called y) has one unit. A set of weights and biases are shown for each ReLU that correctly computes the XOR function.

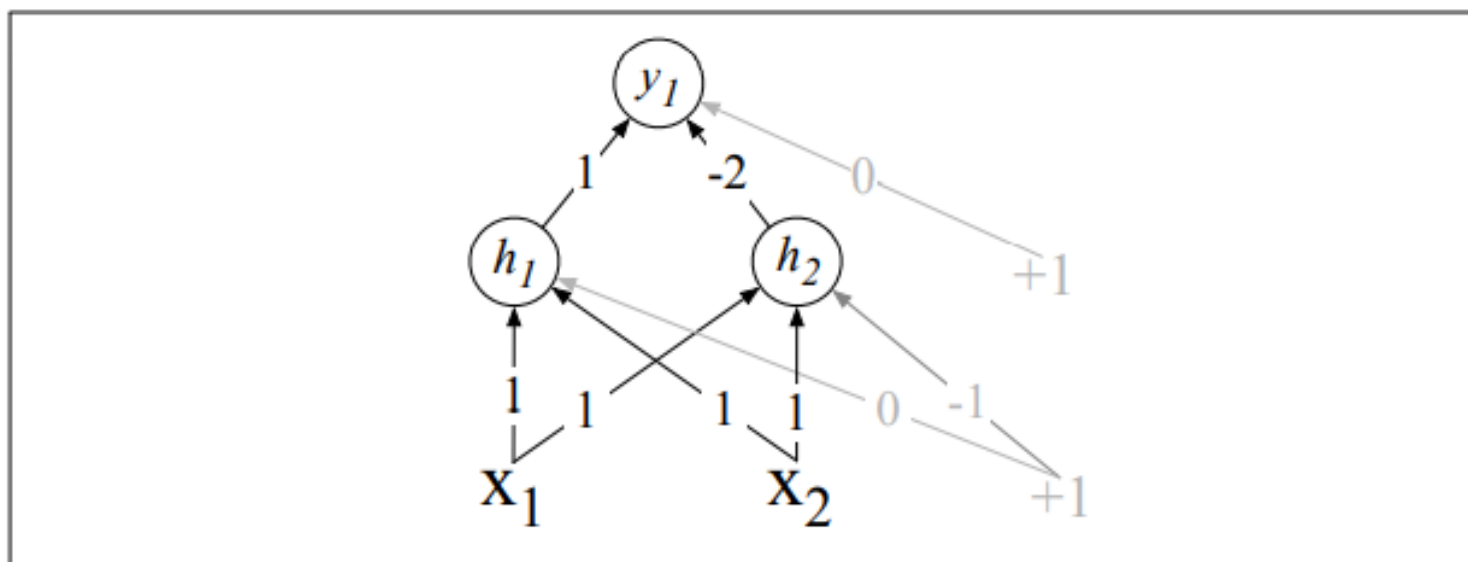


Figure 7.6 XOR solution after [Goodfellow et al. \(2016\)](#). There are three ReLU units, in two layers; we've called them h_1 , h_2 (h for "hidden layer") and y_1 . As before, the numbers on the arrows represent the weights w for each unit, and we represent the bias b as a weight on a unit clamped to $+1$, with the bias weights/units in gray.

The solution: neural networks

- Let's walk through what happens with the input $x = [0 \ 0]$. If we multiply each input value by the appropriate weight, sum, and then add the bias b , we get the vector $[0 \ -1]$, and we then apply the rectified linear transformation to give the output of the h layer as $[0 \ 0]$.
- Now we once again multiply by the weights, sum, and add the bias (0 in this case) resulting in the value 0.
- The reader should work through the computation of the remaining 3 possible input pairs to see that the resulting y values are 1 for the inputs $[0 \ 1]$ and $[1 \ 0]$ and 0 for $[0 \ 0]$ and $[1 \ 1]$.

The solution: neural networks

It's also instructive to look at the intermediate results, the outputs of the two hidden nodes h_1 and h_2 . We showed in the previous paragraph that the h vector for the inputs $x = [0\ 0]$ was $[0\ 0]$. Fig. 7.7b shows the values of the h layer for all 4 inputs. Notice that hidden representations of the two input points $x = [0\ 1]$ and $x = [1\ 0]$ (the two cases with XOR output = 1) are merged to the single point $h = [1\ 0]$. The merger makes it easy to linearly separate the positive and negative cases of XOR. In other words, we can view the hidden layer of the network as forming a representation for the input.

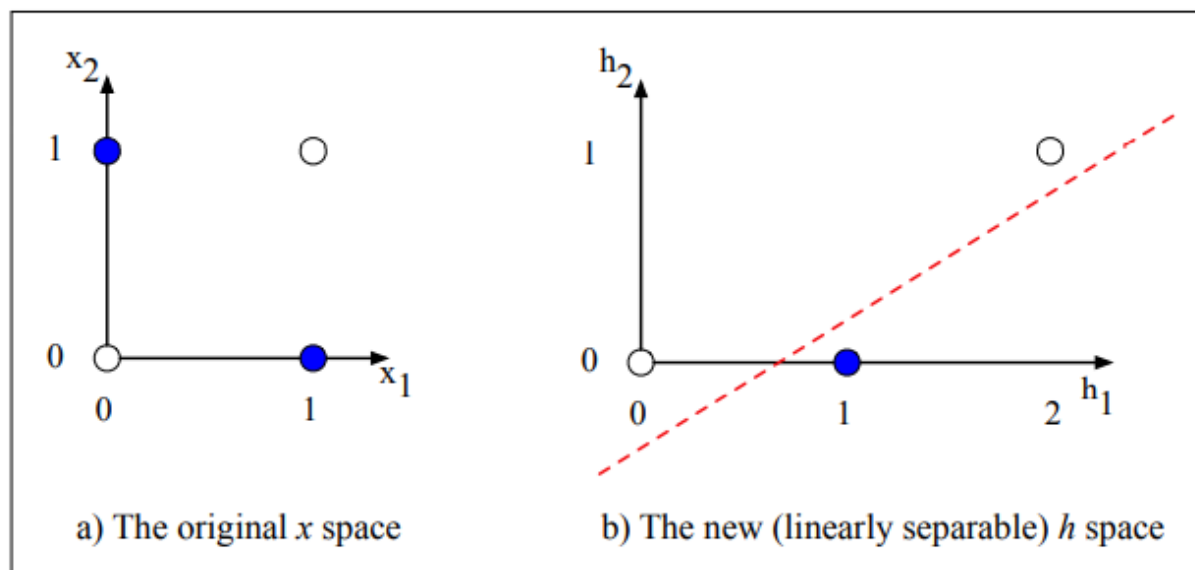


Figure 7.7 The hidden layer forming a new representation of the input. (b) shows the representation of the hidden layer, h , compared to the original input representation x in (a). Notice that the input point $[0\ 1]$ has been collapsed with the input point $[1\ 0]$, making it possible to linearly separate the positive and negative cases of XOR. After Goodfellow et al. (2016).

The solution: neural networks

- In this example we just stipulated the weights in Fig. 7.6. But for real examples the weights for neural networks are learned automatically using the error backpropagation algorithm to be introduced.
- That means the hidden layers will learn to form useful representations. This intuition, that neural networks can automatically learn useful representations of the input, is one of their key advantages.
- Note that the solution to the XOR problem requires a network of units with nonlinear activation functions. A network made up of simple linear (perceptron) units cannot solve the XOR problem.
- This is because a network formed by many layers of purely linear units can always be reduced (i.e., shown to be computationally identical to) a single layer of linear units with appropriate weights, and we've already shown (visually, in Fig. 7.5) that a single unit cannot solve the XOR problem.

Feed-Forward Neural Networks

- Let's now walk through a slightly more formal presentation of the simplest kind of neural network, the feedforward network.
- A feedforward network is a multilayer feedforward network in which the units are connected with no cycles; the outputs from units in each layer are passed to units in the next higher layer, and no outputs are passed back to lower layers.
- For historical reasons multilayer networks, especially feedforward networks, are sometimes called multi-layer perceptrons (or MLPs); this is a technical misnomer, since the units in modern multilayer networks aren't perceptrons (perceptrons are purely linear, but modern networks are made up of units with non-linearities like sigmoids), but at some point the name stuck.

Feed-Forward Neural Networks

The input units are simply scalar values just as we saw in Fig. 7.2.

hidden layer

The core of the neural network is the **hidden layer** formed of **hidden units**, each of which is a neural unit as described in Section 7.1, taking a weighted sum of its inputs and then applying a non-linearity. In the standard architecture, each layer is **fully-connected**, meaning that each unit in each layer takes as input the outputs from all the units in the previous layer, and there is a link between every pair of units from two adjacent layers. Thus each hidden unit sums over all the input units.

fully-connected

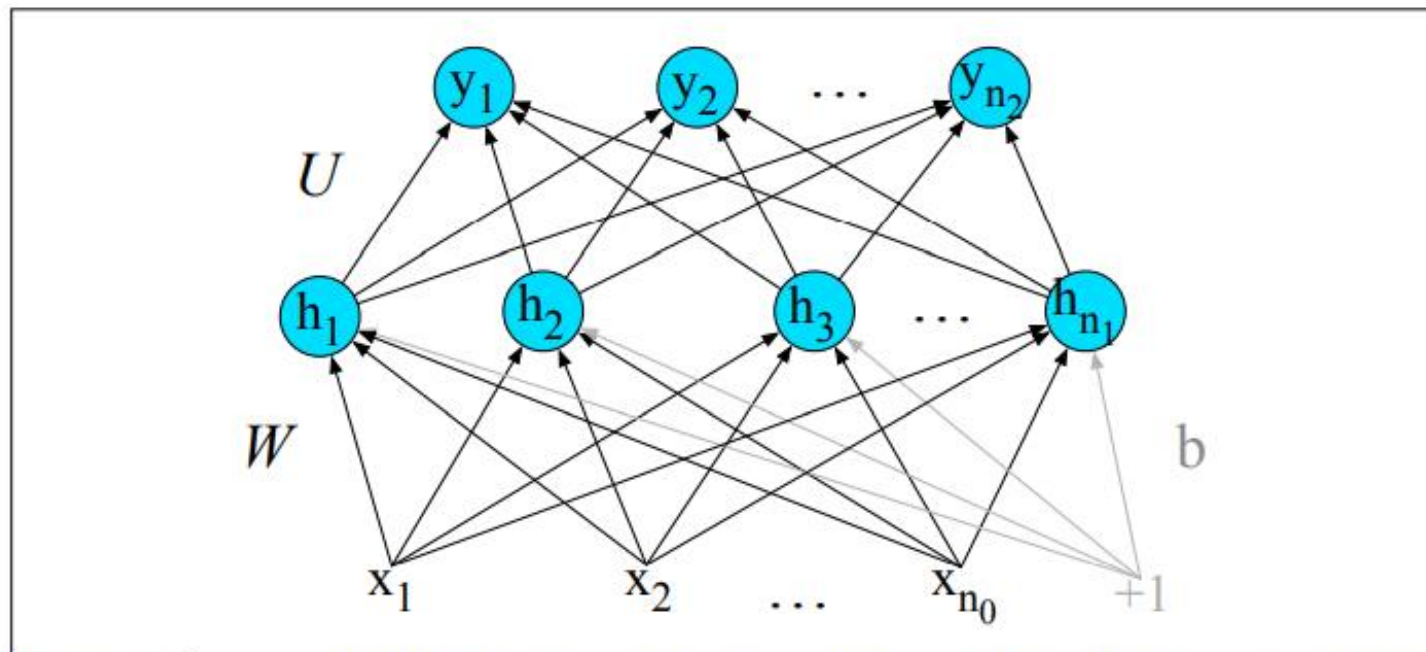


Figure 7.8 A simple 2-layer feedforward network, with one hidden layer, one output layer, and one input layer (the input layer is usually not counted when enumerating layers).

Training Neural Nets

A feedforward neural net is an instance of supervised machine learning in which we know the correct output y for each observation x . What the system produces, via Eq. 7.11, is $\hat{y}$, the system's estimate of the true y . The goal of the training procedure is to learn parameters $W^{[i]}$ and $b^{[i]}$ for each layer i that make $\hat{y}$ for each training observation as close as possible to the true y .

In general, we do all this by drawing on the methods we introduced in Chapter 5 for logistic regression, so the reader should be comfortable with that chapter before proceeding.

First, we'll need a **loss function** that models the distance between the system output and the gold output, and it's common to use the loss function used for logistic regression, the **cross-entropy loss**.

Training Neural Nets

Second, to find the parameters that minimize this loss function, we'll use the **gradient descent** optimization algorithm introduced in Chapter 5.

Third, gradient descent requires knowing the **gradient** of the loss function, the vector that contains the partial derivative of the loss function with respect to each of the parameters. Here is one part where learning for neural networks is more complex than for logistic regression. In logistic regression, for each observation we could directly compute the derivative of the loss function with respect to an individual w or b . But for neural networks, with millions of parameters in many layers, it's much harder to see how to compute the partial derivative of some weight in layer 1 when the loss is attached to some much later layer. How do we partial out the loss over all those intermediate layers?

The answer is the algorithm called **error backpropagation** or **reverse differentiation**.

Computation graphs

A computation graph is a representation of the process of computing a mathematical expression, in which the computation is broken down into separate operations, each of which is modeled as a node in a graph.

Consider computing the function $L(a, b, c) = c(a + 2b)$. If we make each of the component addition and multiplication operations explicit, and add names (d and e) for the intermediate outputs, the resulting series of computations is:

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$

Computation graphs

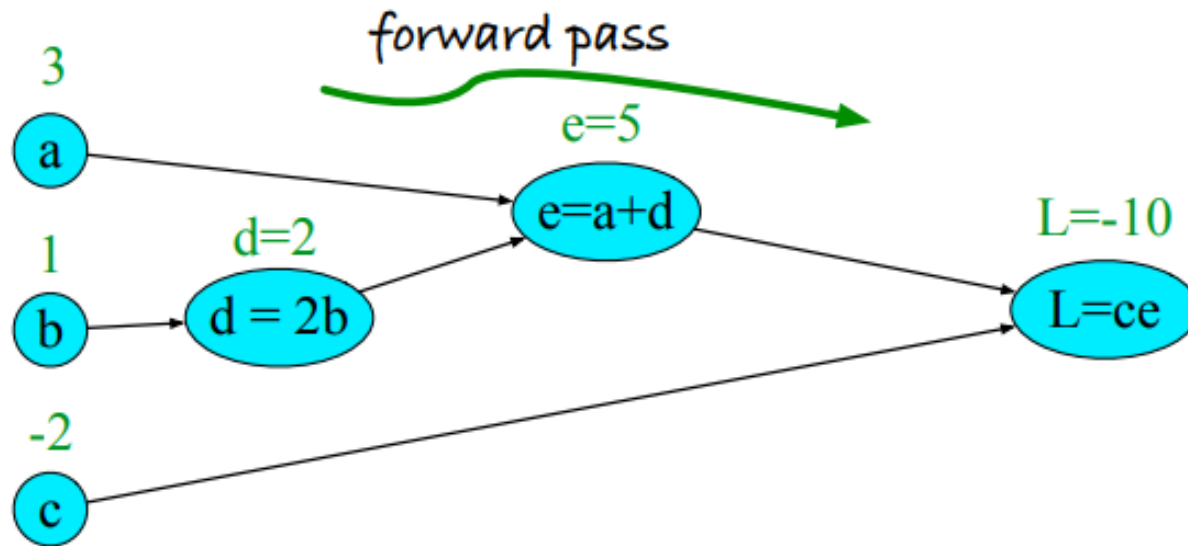


Figure 7.10 Computation graph for the function $L(a, b, c) = c(a + 2b)$, with values for input nodes $a = 3$, $b = 1$, $c = -2$, showing the forward pass computation of L .

Computation graphs

We can now represent this as a graph, with nodes for each operation, and directed edges showing the outputs from each operation as the inputs to the next, as in Fig. 7.10. The simplest use of computation graphs is to compute the value of the function with some given inputs. In the figure, we've assumed the inputs $a = 3$, $b = 1$, $c = -2$, and we've shown the result of the **forward pass** to compute the result $L(3, 1, -2) = -10$. In the forward pass of a computation graph, we apply each operation left to right, passing the outputs of each computation as the input to the next node.

Backward differentiation on computation graphs

The importance of the computation graph comes from the **backward pass**, which is used to compute the derivatives that we'll need for the weight update. In this example our goal is to compute the derivative of the output function L with respect to each of the input variables, i.e., $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$. The derivative $\frac{\partial L}{\partial a}$, tells us how much a small change in a affects L .

chain rule

Backwards differentiation makes use of the **chain rule** in calculus. Suppose we are computing the derivative of a composite function $f(x) = u(v(x))$. The derivative of $f(x)$ is the derivative of $u(x)$ with respect to $v(x)$ times the derivative of $v(x)$ with respect to x :

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx} \quad (7.23)$$

The chain rule extends to more than two functions. If computing the derivative of a composite function $f(x) = u(v(w(x)))$, the derivative of $f(x)$ is:

$$\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx} \quad (7.24)$$

Backward differentiation on computation graphs

Let's now compute the 3 derivatives we need. Since in the computation graph $L = ce$, we can directly compute the derivative $\frac{\partial L}{\partial c}$:

$$\frac{\partial L}{\partial c} = e \quad (7.25)$$

For the other two, we'll need to use the chain rule:

$$\begin{aligned} \frac{\partial L}{\partial a} &= \frac{\partial L}{\partial e} \frac{\partial e}{\partial a} \\ \frac{\partial L}{\partial b} &= \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b} \end{aligned} \quad (7.26)$$

Eq. 7.26 thus requires five intermediate derivatives: $\frac{\partial L}{\partial e}$, $\frac{\partial L}{\partial c}$, $\frac{\partial e}{\partial a}$, $\frac{\partial e}{\partial d}$, and $\frac{\partial d}{\partial b}$, which are as follows (making use of the fact that the derivative of a sum is the sum of the derivatives):

$$\begin{aligned} L = ce & : \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e \\ e = a + d & : \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1 \\ d = 2b & : \quad \frac{\partial d}{\partial b} = 2 \end{aligned}$$

Backward differentiation on computation graphs

In the backward pass, we compute each of these partials along each edge of the graph from right to left, multiplying the necessary partials to result in the final derivative we need. Thus we begin by annotating the final node with $\frac{\partial L}{\partial L} = 1$. Moving to the left, we then compute $\frac{\partial L}{\partial c}$ and $\frac{\partial L}{\partial e}$, and so on, until we have annotated the graph all the way to the input variables. The forward pass conveniently already will have computed the values of the forward intermediate variables we need (like d and e) to compute these derivatives. Fig. 7.11 shows the backward pass. At each node we need to compute the local partial derivative with respect to the parent, multiply it by the partial derivative that is being passed down from the parent, and then pass it to the child.

Backward differentiation on computation graphs

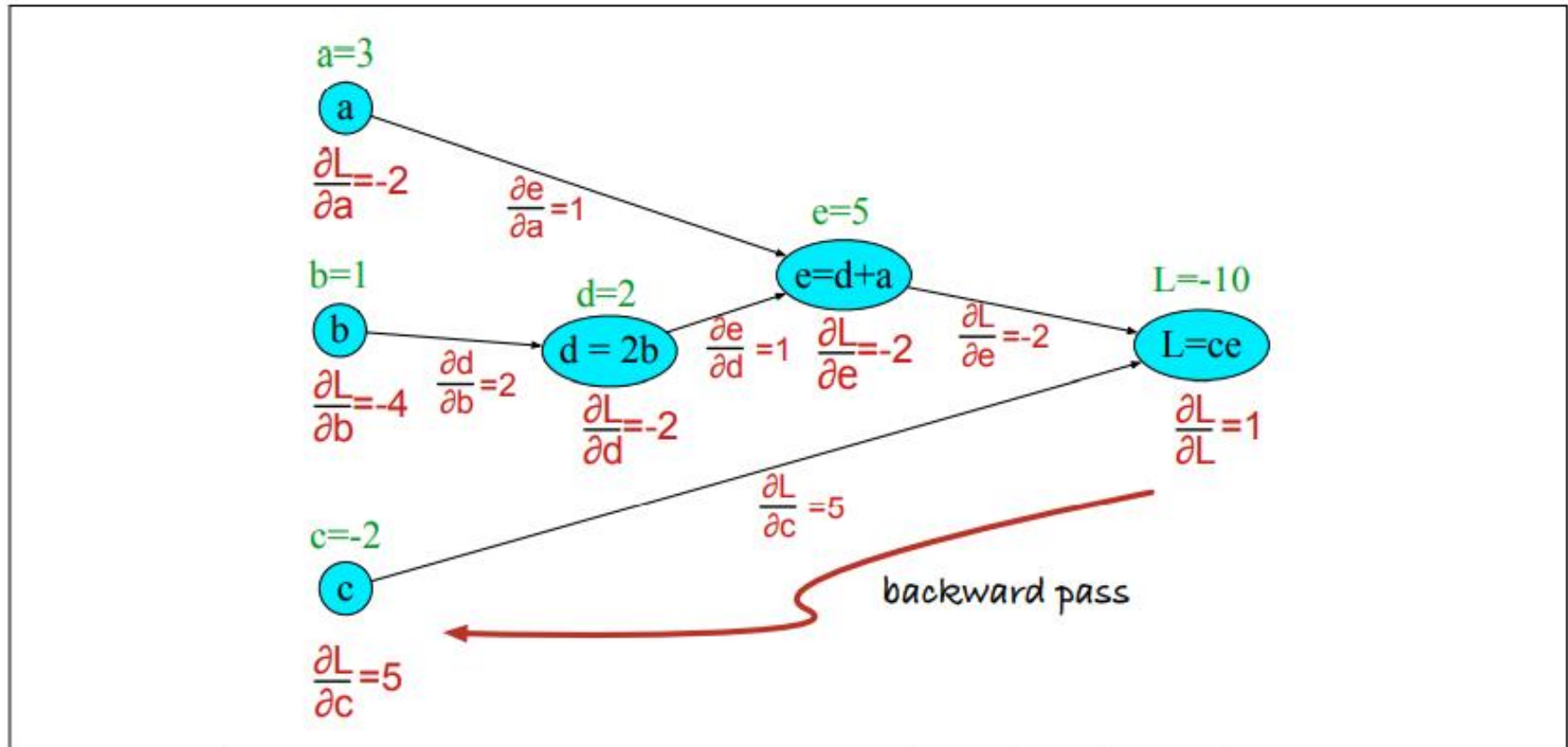


Figure 7.11 Computation graph for the function $L(a, b, c) = c(a + 2b)$, showing the backward pass computation of $\frac{\partial L}{\partial a}$, $\frac{\partial L}{\partial b}$, and $\frac{\partial L}{\partial c}$.

Backward differentiation for a neural network

Of course computation graphs for real neural networks are much more complex. Fig. 7.12 shows a sample computation graph for a 2-layer neural network with $n_0 = 2$, $n_1 = 2$, and $n_2 = 1$, assuming binary classification and hence using a sigmoid output unit for simplicity. The function that the computation graph is computing is:

$$\begin{aligned}z^{[1]} &= W^{[1]}\mathbf{x} + b^{[1]} \\a^{[1]} &= \text{ReLU}(z^{[1]}) \\z^{[2]} &= W^{[2]}a^{[1]} + b^{[2]} \\a^{[2]} &= \sigma(z^{[2]}) \\\hat{y} &= a^{[2]}\end{aligned}\tag{7.27}$$

The weights that need updating (those for which we need to know the partial derivative of the loss function) are shown in orange. In order to do the backward pass, we'll need to know the derivatives of all the functions in the graph. We already saw in Section 5.8 the derivative of the sigmoid σ :

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))\tag{7.28}$$

Backward differentiation for a neural network

We'll also need the derivatives of each of the other activation functions. The derivative of tanh is:

$$\frac{d \tanh(z)}{dz} = 1 - \tanh^2(z) \quad (7.29)$$

The derivative of the ReLU is

$$\frac{d \text{ReLU}(z)}{dz} = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases} \quad (7.30)$$

Backward differentiation for a neural network

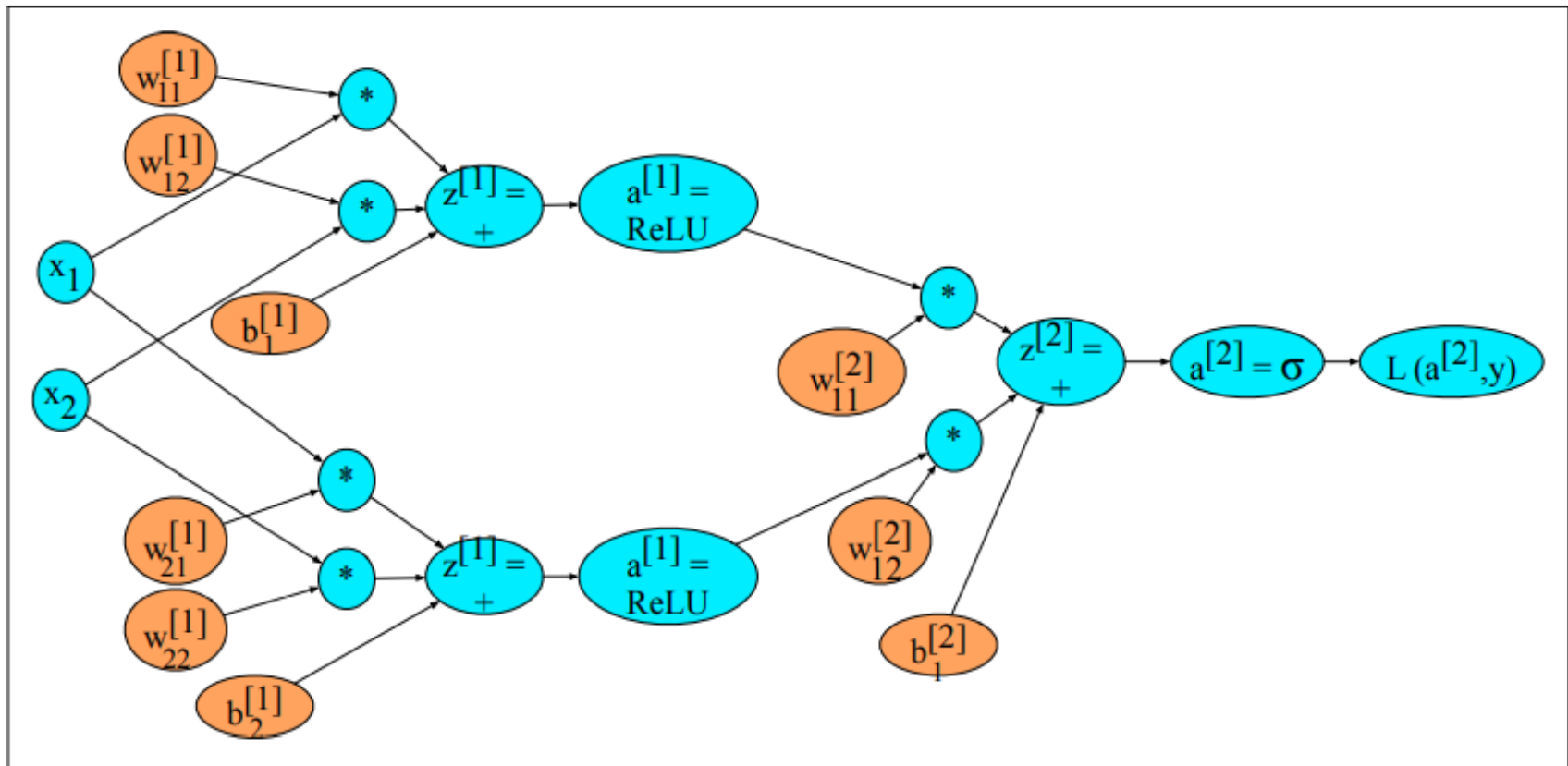
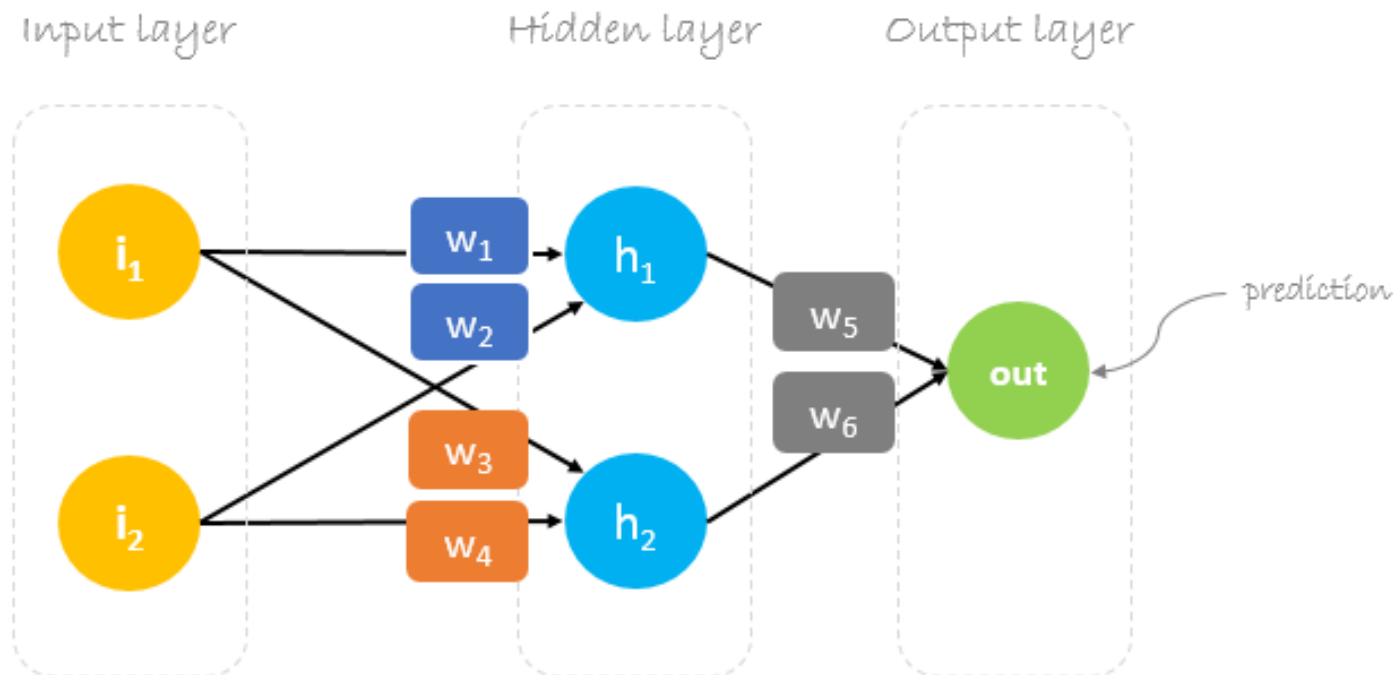


Figure 7.12 Sample computation graph for a simple 2-layer neural net (= 1 hidden layer) with two input dimensions and 2 hidden dimensions.

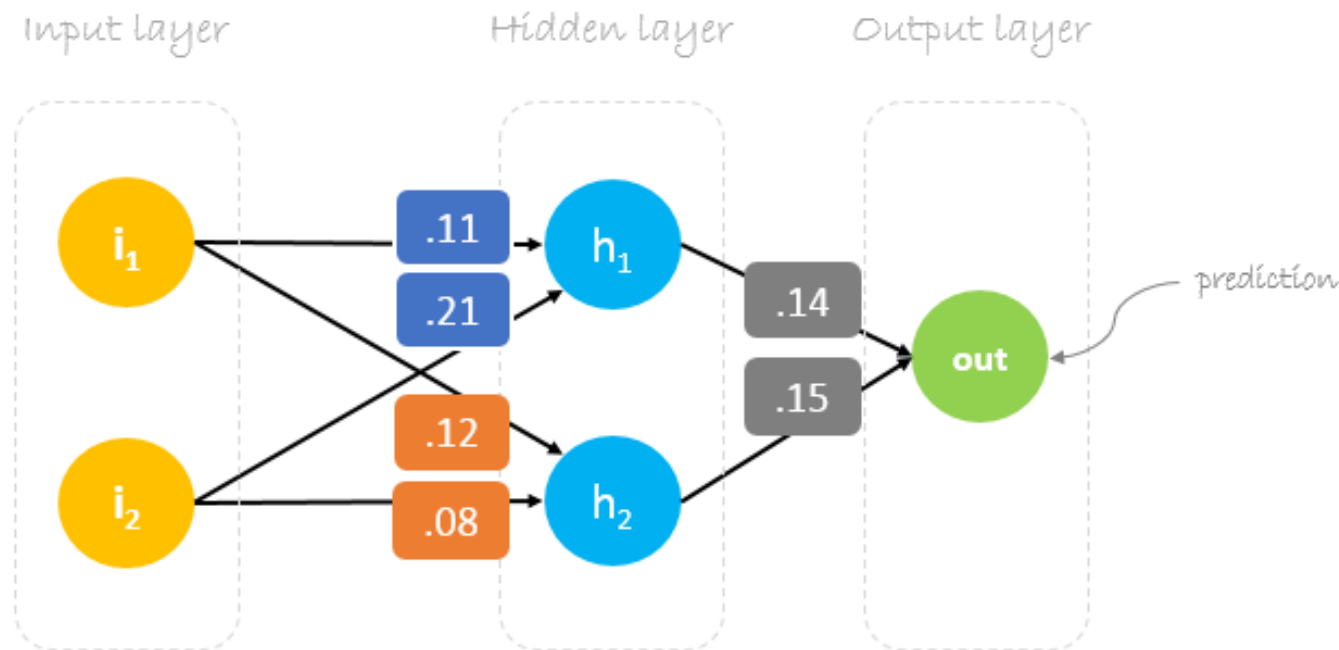
Backpropagation Step by Step: Example

We will build a neural network with three layers:

- **Input** layer with two inputs neurons
- One **hidden** layer with two neurons
- **Output** layer with a single neuron

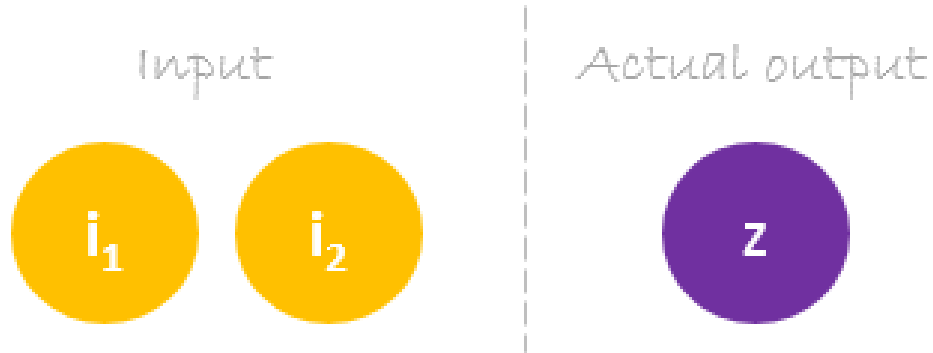


- Neural network training is about finding weights that minimize prediction error. We usually start our training with a set of randomly generated weights. Then, backpropagation is used to update the weights in an attempt to correctly map arbitrary inputs to outputs.
- Our initial weights will be as following: $w_1 = 0.11$, $w_2 = 0.21$, $w_3 = 0.12$, $w_4 = 0.08$, $w_5 = 0.14$ and $w_6 = 0.15$.



Dataset

- Our dataset has one sample with two inputs and one output.

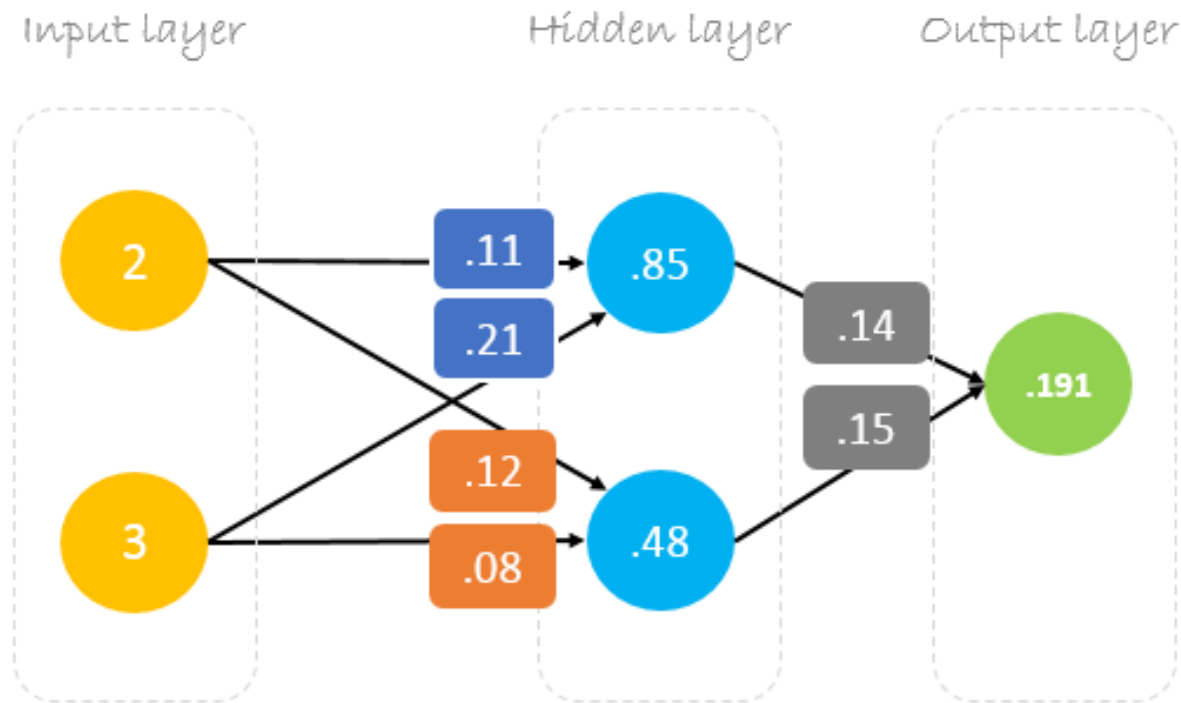


- Our single sample is as following inputs=[2,3] and output=[1].



Forward Pass

- We will use given weights and inputs to predict the output. Inputs are multiplied by weights; the results are then passed forward to next layer.



Forward Pass

$$\begin{bmatrix} 2 & 3 \end{bmatrix} \cdot \begin{bmatrix} 0.11 & 0.12 \\ 0.21 & 0.08 \end{bmatrix} = \begin{bmatrix} 0.85 & 0.48 \end{bmatrix} \cdot \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} = \begin{bmatrix} 0.191 \end{bmatrix}$$

Matrix multiplication

Details

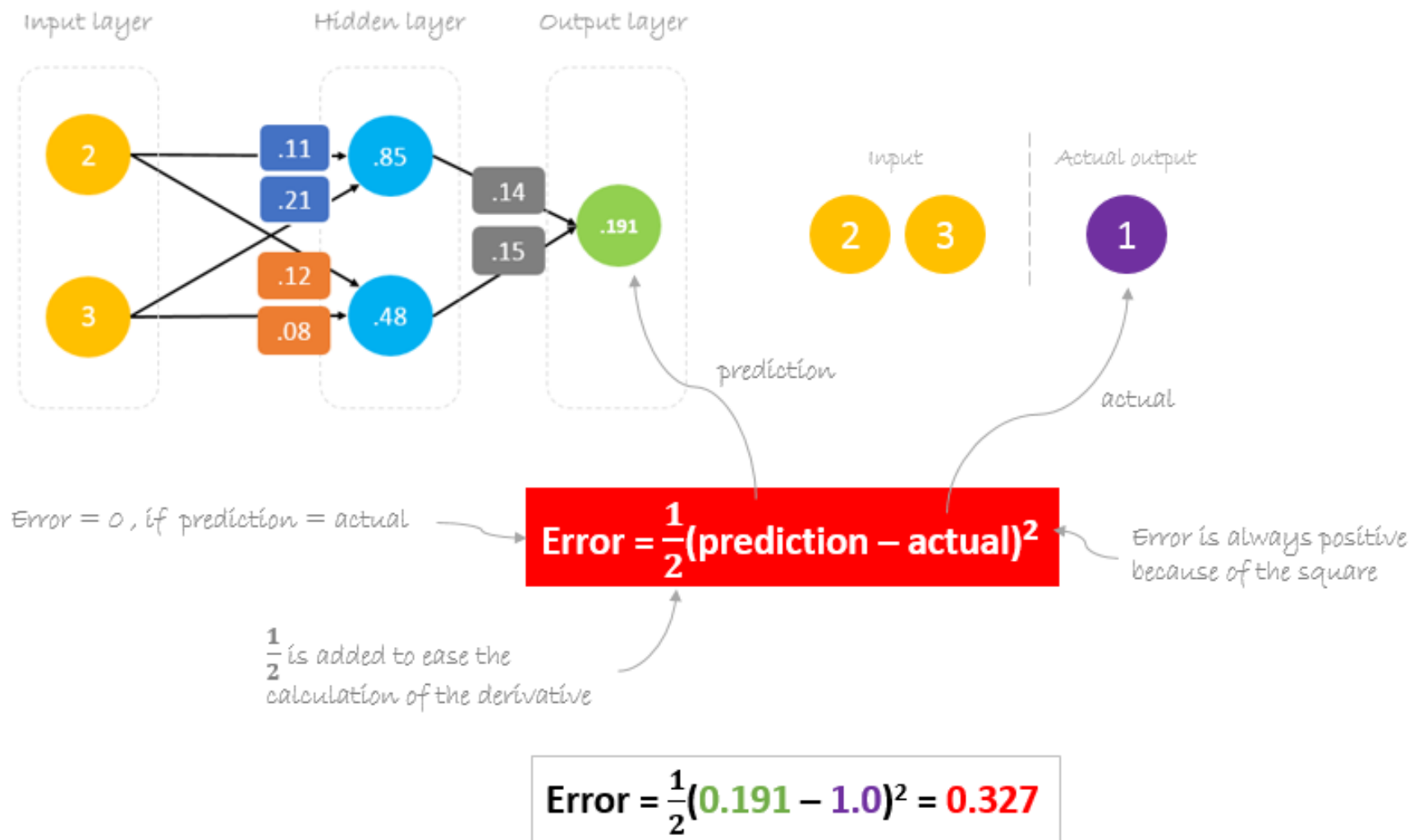
$$2 \times .11 + 3 \times .21 = .85$$

$$.85 \times .14 + .48 \times .15 = .191$$

$$2 \times .12 + 3 \times .08 = .48$$

Calculating Error

- Now, it's time to find out how our network performed by calculating the difference between the actual output and predicted one. It's clear that our network output, or prediction, is not even close to actual output. We can calculate the difference or the error as following.



Reducing Error

- Our main goal of the training is to reduce the **error** or the difference between **prediction** and **actual output**. Since **actual output** is constant, “not changing”, the only way to reduce the error is to change **prediction** value. The question now is, how to change **prediction** value?
- By decomposing **prediction** into its basic elements we can find that **weights** are the variable elements affecting **prediction** value. In other words, in order to change **prediction** value, we need to change **weights** values.

$$\text{prediction} = \text{out}$$



$$\text{prediction} = (h_1) w_5 + (h_2) w_6$$



$$h_1 = i_1 w_1 + i_2 w_2$$

$$h_2 = i_1 w_3 + i_2 w_4$$

$$\text{prediction} = (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$$

to change **prediction** value,
we need to change **weights**

- The question now is how to change/update the weights value so that the error is reduced?
- The answer is Backpropagation!

Backpropagation

- Backpropagation, short for “backward propagation of errors”, is a mechanism used to update the weights using gradient descent. It calculates the gradient of the error function with respect to the neural network’s weights. The calculation proceeds backwards through the network.
- **Gradient descent** is an iterative optimization algorithm for finding the minimum of a function; in our case we want to minimize the error function. To find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient of the function at the current point.

The diagram illustrates the weight update formula for gradient descent. The equation is $*W_x = W_x - a \left(\frac{\partial \text{Error}}{\partial W_x} \right)$. Handwritten annotations identify the components: 'Old weight' points to W_x , 'Derivative of Error with respect to weight' points to $\frac{\partial \text{Error}}{\partial W_x}$, 'New weight' points to $*W_x$, and 'Learning rate' points to the yellow 'a'.

$$*W_x = W_x - a \left(\frac{\partial \text{Error}}{\partial W_x} \right)$$

- For example, to update w_6 , we take the current w_6 and subtract the partial derivative of **error** function with respect to w_6 . Optionally, we multiply the derivative of the **error** function by a selected number to make sure that the new updated **weight** is minimizing the error function; this number is called **learning rate**.

$$*W_6 = W_6 - a \left(\frac{\partial \text{Error}}{\partial W_6} \right)$$

- The derivation of the error function is evaluated by applying the chain rule as following.

$$\begin{aligned} \frac{\partial \text{Error}}{\partial W_6} &= \frac{\partial \text{Error}}{\partial \text{prediction}} * \frac{\partial \text{prediction}}{\partial W_6} \quad \leftarrow \text{chain rule} \\ \text{Error} &= \frac{1}{2}(\text{prediction} - \text{actual})^2 \\ \frac{\partial \text{Error}}{\partial W_6} &= \frac{1}{2}(\text{prediction} - \text{actual})^2 * \frac{\partial (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6}{\partial W_6} \\ \text{prediction} &= (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6 \\ \frac{\partial \text{Error}}{\partial W_6} &= 2 * \frac{1}{2}(\text{prediction} - \text{actual}) \frac{\partial (\text{prediction} - \text{actual})}{\partial \text{prediction}} * (i_1 w_3 + i_2 w_4) \quad \leftarrow h_2 = i_1 w_3 + i_2 w_4 \\ \frac{\partial \text{Error}}{\partial W_6} &= (\text{prediction} - \text{actual}) * (h_2) \quad \leftarrow \Delta = \text{prediction} - \text{actual} \quad \leftarrow \text{delta} \\ \frac{\partial \text{Error}}{\partial W_6} &= \Delta h_2 \end{aligned}$$

- So to update w_6 we can apply the following formula.

$$*W_6 = W_6 - a \Delta h_2$$

- Similarly, we can derive the update formula for w_5 and any other weights existing between the output and the hidden layer.

$$*W_5 = W_5 - a \Delta h_1$$

- However, when moving backward to update w_1 , w_2 , w_3 and w_4 existing between input and hidden layer, the partial derivative for the error function with respect to w_1 , for example, will be as following.

$$\frac{\partial \text{Error}}{\partial W_1} = \frac{\partial \text{Error}}{\partial \text{prediction}} * \frac{\partial \text{prediction}}{\partial h_1} * \frac{\partial h_1}{\partial W_1} \quad \leftarrow \text{chain rule}$$

$$\frac{\partial \text{Error}}{\partial W_1} = \frac{\partial \frac{1}{2}(\text{prediction} - \text{actual})^2}{\partial \text{prediction}} * \frac{\partial (h_1 w_5 + h_2 w_6)}{\partial h_1} * \frac{\partial i_1 w_1 + i_2 w_2}{\partial w_1}$$

$$\frac{\partial \text{Error}}{\partial W_1} = 2 * \frac{1}{2} (\text{prediction} - \text{actual}) \frac{\partial (\text{prediction} - \text{actual})}{\partial \text{prediction}} * (w_5) * (i_1)$$

$$\frac{\partial \text{Error}}{\partial W_1} = (\text{prediction} - \text{actual}) * (w_5 i_1)$$

$$\frac{\partial \text{Error}}{\partial W_1} = \Delta w_5 i_1$$

$$\text{Error} = \frac{1}{2} (\text{prediction} - \text{actual})^2$$

$$\text{prediction} = (h_1) w_5 + (h_2) w_6$$

$$h_1 = i_1 w_1 + i_2 w_2$$

$$\Delta = \text{prediction} - \text{actual}$$

$\leftarrow$ delta

- We can find the update formula for the remaining weights w_2 , w_3 and w_4 in the same way.
- In summary, the update formulas for all weights will be as following:

updated weights


$$\begin{aligned} *w_6 &= w_6 - a (h_2 \cdot \Delta) \\ *w_5 &= w_5 - a (h_1 \cdot \Delta) \\ *w_4 &= w_4 - a (i_2 \cdot \Delta w_6) \\ *w_3 &= w_3 - a (i_1 \cdot \Delta w_6) \\ *w_2 &= w_2 - a (i_2 \cdot \Delta w_5) \\ *w_1 &= w_1 - a (i_1 \cdot \Delta w_5) \end{aligned}$$

- We can rewrite the update formulas in matrices as following.

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a \Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} ah_1\Delta \\ ah_2\Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a \Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} \cdot [w_5 \quad w_6] = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} ai_1\Delta w_5 & ai_1\Delta w_6 \\ ai_2\Delta w_5 & ai_2\Delta w_6 \end{bmatrix}$$

Backward Pass

- Using derived formulas we can find the new **weights**.
- **Learning rate**: is a hyperparameter which means that we need to manually guess its value.

$$\Delta = 0.191 - 1 = -0.809 \quad \leftarrow \text{Delta} = \text{prediction} - \text{actual}$$

$$a = 0.05 \quad \leftarrow \text{Learning rate, we smartly guess this number}$$

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} \cdot [0.14 \quad 0.15] = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$

- Now, using the new **weights** we will repeat the forward pass.
- We can notice that the **prediction** 0.26 is a little bit closer to **actual output** than the previously predicted one 0.191. We can repeat the same process of backward and forward pass until **error** is close or equal to zero.

